

In []: Nmae: Aditi Vishwanath Sadalkar
Roll_No. C02110

Pandas Complete Course Notebook

```
In [13]: import numpy as np
import pandas as pd

print("pandas version:", pd.__version__)
print("numpy version:", np.__version__)
```

pandas version: 2.3.1
numpy version: 1.26.0

1. Conceptual Foundations

```
In [ ]: # Quick illustration: same data, four containers

data = ['Apple', 2, 3.4]

py_list = data
py_dict = {'item': 'apple', 'count': 2, 'price': 3.4}
np_arr = np.array(data, dtype=object)
pd_series = pd.Series(data, index=['item', 'count', 'price'])

print("list:", py_list)
print("dict:", py_dict)
print("numpy array:", np_arr)
print("\n pandas series (labeled, mixed types):")
print(pd_series)
```

2. Pandas Series

```
In [14]: # Create a series from a List(default integer index)

lst = ['apple', 3]
s = pd.Series(lst)
print(s)
print(type(s))
```

```
0    apple
1         3
dtype: object
<class 'pandas.core.series.Series'>
```

```
In [15]: # Create series with custom index
lst = ['cherry', 3]
s = pd.Series(lst, index=['a', 'c'])
print(s)
```

```
a    cherry
c         3
dtype: object
```

In [17]: *# Create a series from a numpy array*

```
a = np.array(['cherry', 44])
print(a, type(a))

s = pd.Series(a, index=['a', 'b'])
print(s)
```

```
['cherry' '44'] <class 'numpy.ndarray'>
a    cherry
b         44
dtype: object
```

In [16]: *# Create a series from a dictionary -> keys become the index*

```
import pandas as pd
d = {'a': [1,2,3], 'b': [3,2,1]}
s = pd.Series(d)
print(s)
```

```
a    [1, 2, 3]
b    [3, 2, 1]
dtype: object
```

In [19]: *# Indexing into a series*

```
s = pd.Series([10,20,30,40], index=['w', 'x', 'y', 'z'])
print("by label:", s['x'])
print("by position:", s.iloc[1])
print("slice by label:\n", s['x' : 'z']) # label slicing is INCLUSIVE
print("slice by position:\n", s.iloc[1:3]) # positional slicing is EXCLUSIVE of
```

```
by label: 20
by position: 20
slice by label:
x    20
y    30
z    40
dtype: int64
slice by position:
x    20
y    30
dtype: int64
```

3. DataFrame Creation

In [17]: *# Create DataFrame from a dict*

```
s = {'col1': [1,2]}
df_dict = pd.DataFrame(s)
print(df_dict)
print(type(df_dict))
print("shape:", df_dict.shape)
```

```
col1
0    1
1    2
<class 'pandas.core.frame.DataFrame'>
shape: (2, 1)
```

In [18]: *# create DataFrame from a nested list (must supply column name)*

```
data = [[1,2,3], [5,3,2]]
```

```
df_list = pd.DataFrame (data,columns=['col1', 'col2', 'col3'])
df_list
```

Out[18]:

	col1	col2	col3
0	1	2	3
1	5	3	2

In [19]: *# Create DataFrame from a 2D NumPy array*

```
import numpy as np
arr = np.array([[1,2,3],[5,3,2]])
df_arr = pd.DataFrame (arr,columns=['col1', 'col2', 'col3'])
df_arr
```

Out[19]:

	col1	col2	col3
0	1	2	3
1	5	3	2

In [20]: *# Create DataFrame from a dictionary of equal-length lists*

```
df_multi = pd.DataFrame({
    'name': ['Anvi', 'Umangi', 'Shravani'],
    'age': [20,30,25],
    'score': [80.3,75.00,83.4]
})
df_multi
```

Out[20]:

	name	age	score
0	Anvi	20	80.3
1	Umangi	30	75.0
2	Shravani	25	83.4

4. Combining DataFrames

In [21]:

```
import pandas as pd
temp_df = pd.DataFrame({'city': ['mum', 'del', 'bang', 'hyd'], 'temp': [30,43,35
hum_df = pd.DataFrame({'city': ['mum', 'del', 'hyd','chen'], 'humidity': [65,63,

print(temp_df)
print()
print(hum_df)
```

	city	temp
0	mum	30
1	del	43
2	bang	35
3	hyd	36

	city	humidity
0	mum	65
1	del	63
2	hyd	66
3	chen	65

```
In [22]: # concat: stacks rows on top of each other (note duplicate index unless ignore_i
pd.concat([temp_df, hum_df], ignore_index=True)
```

```
Out[22]:
```

	city	temp	humidity
0	mum	30.0	NaN
1	del	43.0	NaN
2	bang	35.0	NaN
3	hyd	36.0	NaN
4	mum	NaN	65.0
5	del	NaN	63.0
6	hyd	NaN	66.0
7	chen	NaN	65.0

```
In [23]: # concat along columns (axis=1) - aligns by index position
pd.concat([temp_df, hum_df], axis=1)
```

```
Out[23]:
```

	city	temp	city	humidity
0	mum	30	mum	65
1	del	43	del	63
2	bang	35	hyd	66
3	hyd	36	chen	65

```
In [24]: # merge: outer join keeps all keys from both sides, fillings NaN where unmuted
pd.merge(temp_df, hum_df, on='city', how='outer')
```

Out[24]:

	city	temp	humidity
0	bang	35.0	NaN
1	chen	NaN	65.0
2	del	43.0	63.0
3	hyd	36.0	66.0
4	mum	30.0	65.0

In [25]: *# merge: inner join keeps ONLY keys present in both DataFrames*

```
pd.merge(temp_df, hum_df, on='city', how='inner')
```

Out[25]:

	city	temp	humidity
0	mum	30	65
1	del	43	63
2	hyd	36	66

In [26]: *# merge: left \ right joins*

```
print(pd.merge(temp_df, hum_df, on='city', how='left'))
print()
print(pd.merge(temp_df, hum_df, on='city', how='right'))
```

	city	temp	humidity
0	mum	30	65.0
1	del	43	63.0
2	bang	35	NaN
3	hyd	36	66.0

	city	temp	humidity
0	mum	30.0	65
1	del	43.0	63
2	hyd	36.0	66
3	chen	NaN	65

In [27]: *# join: merges on the INDEX insted of a column*

```
left = temp_df.set_index('city')
right = hum_df.set_index('city')
left.join(right, how='outer')
```

Out[27]:

	temp	humidity
city		
bang	35.0	NaN
chen	NaN	65.0
del	43.0	63.0
hyd	36.0	66.0
mum	30.0	65.0

5. Importing / Exporting Data

```
In [28]: import numpy as np
import pandas as pd
np.random.seed(42)
n = 200

df = pd.DataFrame({
    'Age': np.random.randint(4,91,n),
    'Gender': np.random.choice(['male', 'female'], n, p = [0.75, 0.25]),
    'TB' : np.round(np.random.exponential(2,n),1),
    'DB' : np.round(np.random.exponential(1,n),2),
    'Alkpos' : np.random.randint(60,750,n),
    'sgpt' : np.random.randint(10,300,n),
    'sgot' : np.random.randint(10,350,n),
    'TP' : np.round(np.random.normal(6.5,1.0,n),1),
    'ALB' : np.round(np.random.normal(3.1,0.8,n),1),
    'AG' : np.round(np.random.normal(1.0,0.4,n),2),
    'LiverPatient' : np.random.choice([0,1], n, p=[0.3,0.7])
})

# Inject few missing values and duplicate rows to make later sections meaningful
df.loc[[3,17,42,88], 'AG'] = np.nan
df = pd.concat([df, df.iloc[[5,5,20]]], ignore_index=True)

print(df.shape)
df.head()
```

(203, 11)

Out[28]:

	Age	Gender	TB	DB	Alkpos	sgpt	sgot	TP	ALB	AG	LiverPatient
0	55	male	5.8	1.57	559	243	157	7.1	3.9	0.41	1
1	18	male	5.8	0.89	74	204	72	6.9	2.8	1.07	1
2	75	male	4.0	0.65	682	128	319	7.1	1.1	0.99	0
3	64	male	2.0	0.55	728	29	252	7.8	4.9	NaN	1
4	24	male	3.2	1.54	348	270	339	6.7	2.0	1.05	0

6. DataFrame Attributes

```
In [29]: print("shape:", df.shape)    #(row,column)
         print("size:", df.size)      # total number of cell
         print("nddim:", df.ndim)     # number of dimensions
```

```
shape: (203, 11)
size: 2233
nddim: 2
```

```
In [30]: df.dtypes    # data type every column
```

```
Out[30]: Age                int32
         Gender             object
         TB                 float64
         DB                 float64
         Alkpos             int32
         sgpt               int32
         sgot               int32
         TP                 float64
         ALB                 float64
         AG                 float64
         LiverPatient       int32
         dtype: object
```

```
In [31]: print("columns:", list(df.columns))
         print("index:", df.index)
```

```
columns: ['Age', 'Gender', 'TB', 'DB', 'Alkpos', 'sgpt', 'sgot', 'TP', 'ALB', 'A
G', 'LiverPatient']
index: RangeIndex(start=0, stop=203, step=1)
```

```
In [32]: df.values[:3]    # underlying numpy representation
```

```
Out[32]: array([[55, 'male', 5.8, 1.57, 559, 243, 157, 7.1, 3.9, 0.41, 1],
                [18, 'male', 5.8, 0.89, 74, 204, 72, 6.9, 2.8, 1.07, 1],
                [75, 'male', 4.0, 0.65, 682, 128, 319, 7.1, 1.1, 0.99, 0]],
              dtype=object)
```

```
In [33]: df.axes    # [row index, column index] as list
```

```
Out[33]: [RangeIndex(start=0, stop=203, step=1),
         Index(['Age', 'Gender', 'TB', 'DB', 'Alkpos', 'sgpt', 'sgot', 'TP', 'ALB',
               'AG', 'LiverPatient'],
               dtype='object')]
```

```
In [34]: df.keys    # same as df.column for a dataframe
```

```
Out[34]: <bound method NDFrame.keys of
TP  ALB    AG  LiverPatient  Age  Gender  TB    DB  Alkpos  sgpt  sgot
0    55   male  5.8  1.57    559   243   157  7.1  3.9  0.41      1
1    18   male  5.8  0.89     74   204    72  6.9  2.8  1.07      1
2    75   male  4.0  0.65    682   128   319  7.1  1.1  0.99      0
3    64   male  2.0  0.55    728    29   252  7.8  4.9  NaN       1
4    24   male  3.2  1.54    348   270   339  6.7  2.0  1.05      0
..    ...   ...   ...   ...   ...   ...   ...   ...   ...   ...   ...
198  47   male  6.9  1.31    573    68   318  5.8  2.7  1.37      1
199  35   male  0.2  1.18    582    81    17  8.8  3.4  0.68      1
200  86  female  2.3  1.02    537   265   116  7.2  1.8  0.61      0
201  86  female  2.3  1.02    537   265   116  7.2  1.8  0.61      0
202  36   male  2.1  1.20    177   218    29  4.9  2.3  0.76      1

[203 rows x 11 columns]>
```

```
In [35]: df.info() # dtypes + non-null counts + memory usages
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 203 entries, 0 to 202
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Age              203 non-null    int32
1   Gender           203 non-null    object
2   TB               203 non-null    float64
3   DB               203 non-null    float64
4   Alkpos           203 non-null    int32
5   sgpt             203 non-null    int32
6   sgot             203 non-null    int32
7   TP              203 non-null    float64
8   ALB              203 non-null    float64
9   AG              199 non-null    float64
10  LiverPatient     203 non-null    int32
dtypes: float64(5), int32(5), object(1)
memory usage: 13.6+ KB
```

```
In [36]: df.describe() # summary statistic for numeric column(count,men,std,min,max)
```

```
Out[36]:
```

	Age	TB	DB	Alkpos	sgpt	sgot	
count	203.000000	203.000000	203.000000	203.000000	203.000000	203.000000	203.000000
mean	45.389163	2.040394	1.000591	389.433498	155.320197	171.369458	6.623611
std	25.893179	1.990881	0.908387	202.125878	85.254488	102.553953	1.010311
min	4.000000	0.000000	0.000000	60.000000	12.000000	12.000000	3.600000
25%	24.000000	0.600000	0.405000	215.500000	79.500000	70.500000	6.000000
50%	47.000000	1.400000	0.780000	377.000000	161.000000	177.000000	6.700000
75%	65.000000	2.700000	1.320000	543.500000	231.500000	259.000000	7.300000
max	90.000000	11.000000	5.900000	748.000000	298.000000	347.000000	8.900000

```
In [37]: df.describe(include='object') # summary stats object columns
```


Out[37]:

Gender	
count	203
unique	2
top	male
freq	153

7.Selecting & Indexing

In [38]: `df.iloc[0]` *# first row by position*

Out[38]:

Age	55
Gender	male
TB	5.8
DB	1.57
Alkpos	559
sgpt	243
sgot	157
TP	7.1
ALB	3.9
AG	0.41
LiverPatient	1

Name: 0, dtype: object

In [39]: `df.iloc[-1]` *# last row*

Out[39]:

Age	36
Gender	male
TB	2.1
DB	1.2
Alkpos	177
sgpt	218
sgot	29
TP	4.9
ALB	2.3
AG	0.76
LiverPatient	1

Name: 202, dtype: object

In [40]: `df.iloc[[1, 6]]` *# specific rows by position*

Out[40]:

	Age	Gender	TB	DB	Alkpos	sgpt	sgot	TP	ALB	AG	LiverPatient
1	18	male	5.8	0.89	74	204	72	6.9	2.8	1.07	1
6	90	male	1.7	1.63	290	231	235	6.4	3.9	1.48	1

In [41]: `df.iloc[0:3, 0:2]` *# rows 0-2, columns 0-1*

Out[41]:

	Age	Gender
0	55	male
1	18	male
2	75	male

In [42]: `df.loc[0]` *# row with index label 0*

Out[42]:

Age	55
Gender	male
TB	5.8
DB	1.57
Alkpos	559
sgpt	243
sgot	157
TP	7.1
ALB	3.9
AG	0.41
LiverPatient	1

Name: 0, dtype: object

In [43]: `df.loc[[2,3]]` *# row with index label with 2 and 3*

Out[43]:

	Age	Gender	TB	DB	Alkpos	sgpt	sgot	TP	ALB	AG	LiverPatient
2	75	male	4.0	0.65	682	128	319	7.1	1.1	0.99	0
3	64	male	2.0	0.55	728	29	252	7.8	4.9	NaN	1

In [44]: `df.loc[:, 'Age']`

Out[44]:

0	55
1	18
2	75
3	64
4	24
	..
198	47
199	35
200	86
201	86
202	36

Name: Age, Length: 203, dtype: int32

In [45]: `df.loc[[2,3], 'Age' : 'TP']` *# label based slice*

Out[45]:

	Age	Gender	TB	DB	Alkpos	sgpt	sgot	TP
2	75	male	4.0	0.65	682	128	319	7.1
3	64	male	2.0	0.55	728	29	252	7.8

In [46]: `df.loc[:, ['TB', 'DB']]` *# multiple specific column*

```
Out[46]:
```

	TB	DB
0	5.8	1.57
1	5.8	0.89
2	4.0	0.65
3	2.0	0.55
4	3.2	1.54
...	...	...
198	6.9	1.31
199	0.2	1.18
200	2.3	1.02
201	2.3	1.02
202	2.1	1.20

203 rows × 2 columns

```
In [47]: # Boolean indexing
male_data = df[df['Gender'] == 'male']
male_data.head()
```

```
Out[47]:
```

	Age	Gender	TB	DB	Alkpos	sgpt	sgot	TP	ALB	AG	LiverPatient
0	55	male	5.8	1.57	559	243	157	7.1	3.9	0.41	1
1	18	male	5.8	0.89	74	204	72	6.9	2.8	1.07	1
2	75	male	4.0	0.65	682	128	319	7.1	1.1	0.99	0
3	64	male	2.0	0.55	728	29	252	7.8	4.9	NaN	1
4	24	male	3.2	1.54	348	270	339	6.7	2.0	1.05	0

```
In [48]: # multiple conditions: use |, &, / with parentheses( NOT 'and' / 'or')

df[(df['Age'] >= 50) & (df['Gender'] == 'Male')]
df.head()
```

```
Out[48]:
```

	Age	Gender	TB	DB	Alkpos	sgpt	sgot	TP	ALB	AG	LiverPatient
0	55	male	5.8	1.57	559	243	157	7.1	3.9	0.41	1
1	18	male	5.8	0.89	74	204	72	6.9	2.8	1.07	1
2	75	male	4.0	0.65	682	128	319	7.1	1.1	0.99	0
3	64	male	2.0	0.55	728	29	252	7.8	4.9	NaN	1
4	24	male	3.2	1.54	348	270	339	6.7	2.0	1.05	0

```
In [49]: # .query() - same filters more reliable for complex condition
df.query("Age >= 50 and Gender == 'Male'")
```

```
df.head()
```

```
Out[49]:
```

	Age	Gender	TB	DB	Alkpos	sgpt	sgot	TP	ALB	AG	LiverPatient
0	55	male	5.8	1.57	559	243	157	7.1	3.9	0.41	1
1	18	male	5.8	0.89	74	204	72	6.9	2.8	1.07	1
2	75	male	4.0	0.65	682	128	319	7.1	1.1	0.99	0
3	64	male	2.0	0.55	728	29	252	7.8	4.9	NaN	1
4	24	male	3.2	1.54	348	270	339	6.7	2.0	1.05	0

8. Sorting, Ranking, Rename, Drop, Insert, Assign, Replace

```
In [50]: # sort by a single column
df.sort_values(by='DB', ascending=False).head()
```

```
Out[50]:
```

	Age	Gender	TB	DB	Alkpos	sgpt	sgot	TP	ALB	AG	LiverPatient
121	50	male	5.0	5.90	535	277	83	7.4	2.9	0.83	0
165	80	male	0.9	4.70	502	150	84	5.4	3.5	0.62	0
66	57	male	3.0	4.29	521	158	233	8.2	3.0	0.58	0
9	27	female	3.4	3.94	609	289	250	8.3	3.6	0.63	1
126	17	female	0.5	3.69	200	297	344	6.9	2.5	1.09	1

```
In [51]: # sort by multiple column
df.sort_values(by=['Gender', 'Age'], ascending=[True, False]).head()
```

```
Out[51]:
```

	Age	Gender	TB	DB	Alkpos	sgpt	sgot	TP	ALB	AG	LiverPatient
5	86	female	2.3	1.02	537	265	116	7.2	1.8	0.61	0
200	86	female	2.3	1.02	537	265	116	7.2	1.8	0.61	0
201	86	female	2.3	1.02	537	265	116	7.2	1.8	0.61	0
83	85	female	4.4	0.11	261	248	68	6.0	2.9	1.47	1
58	84	female	1.3	1.16	383	131	46	5.8	2.6	1.02	0

```
In [52]: # rank() -assigns a rank each value

df['Age_rank'] = df['Age'].rank(ascending=False)
df[['Age', 'Age_rank']].head()
```

Out[52]:

	Age	Age_rank
0	55	83.5
1	18	159.5
2	75	33.5
3	64	60.0
4	24	152.0

In [53]: *# Renmae columns*
df_renamed = df.rename(columns={'TB': 'Total_Bilirubin', 'DB': 'Direct_Bilirubin'})
df_renamed.head(2)

Out[53]:

	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkpos	sgpt	sgot	TP	ALB	AG	Li
0	55	male	5.8	1.57	559	243	157	7.1	3.9	0.41	
1	18	male	5.8	0.89	74	204	72	6.9	2.8	1.07	

◀  ▶

In [54]: *# Drop columns*
df.drop('Age_rank', axis=1, inplace=True)
df.drop(['sgpt', 'sgot'], axis=1).head(2)

Out[54]:

	Age	Gender	TB	DB	Alkpos	TP	ALB	AG	LiverPatient
0	55	male	5.8	1.57	559	7.1	3.9	0.41	1
1	18	male	5.8	0.89	74	6.9	2.8	1.07	1

In [55]: *# drop rows*
df.drop([0,1,2], axis=0).head()

Out[55]:

	Age	Gender	TB	DB	Alkpos	sgpt	sgot	TP	ALB	AG	LiverPatient
3	64	male	2.0	0.55	728	29	252	7.8	4.9	NaN	1
4	24	male	3.2	1.54	348	270	339	6.7	2.0	1.05	0
5	86	female	2.3	1.02	537	265	116	7.2	1.8	0.61	0
6	90	male	1.7	1.63	290	231	235	6.4	3.9	1.48	1
7	78	female	0.3	2.33	479	235	322	7.9	5.1	0.94	0

In [56]: *# Insert a column at a specific position*
df_copy = df.copy()
df_copy.insert(1, 'AgeGroup', np.where(df_copy['Age'] >=50, 'senior', 'adult'))
df_copy.head(3)

Out[56]:

	Age	AgeGroup	Gender	TB	DB	Alkpos	sgpt	sgot	TP	ALB	AG	LiverPatient
0	55	senior	male	5.8	1.57	559	243	157	7.1	3.9	0.41	1
1	18	adult	male	5.8	0.89	74	204	72	6.9	2.8	1.07	1
2	75	senior	male	4.0	0.65	682	128	319	7.1	1.1	0.99	0

In [57]: *# assign() - create one or more new columns in chainable / functional style*

```
df_assigned = df.assign(
    TB_per_DB=lambdax : x['TB'] / (x['DB'] + 0.01),
    IsMale=lambdax : x['Gender'] == 'Male'
)
df_assigned.head(3)
```

Out[57]:

	Age	Gender	TB	DB	Alkpos	sgpt	sgot	TP	ALB	AG	LiverPatient	TB_per_DB
0	55	male	5.8	1.57	559	243	157	7.1	3.9	0.41	1	3.670886
1	18	male	5.8	0.89	74	204	72	6.9	2.8	1.07	1	6.444444
2	75	male	4.0	0.65	682	128	319	7.1	1.1	0.99	0	6.060606

In [58]: *# replace() - swap specific values*

```
df['Gender'].replace({'Male': 'M', 'Female': 'F'}).head()
```

Out[58]:

```
0    male
1    male
2    male
3    male
4    male
Name: Gender, dtype: object
```

9. map, apply, DataFrame.map and Lambda Function

In [59]: *# map() on series with dictionary*

```
gender_code = {'Male': 1, 'Female': 0}
df['Gender_code'] = df['Gender'].map(gender_code)
df[['Gender', 'Gender_code']].head()
```

Out[59]:

	Gender	Gender_code
0	male	NaN
1	male	NaN
2	male	NaN
3	male	NaN
4	male	NaN

In [60]: *# map() with a function / Lambda*

```
df['Age_in_months'] = df['Age'].map(lambdax : x * 12)
```

```
df[['Age', 'Age_in_months']]
df.head()
```

Out[60]:

	Age	Gender	TB	DB	Alkpos	sgpt	sgot	TP	ALB	AG	LiverPatient	Gender_cc
0	55	male	5.8	1.57	559	243	157	7.1	3.9	0.41	1	N
1	18	male	5.8	0.89	74	204	72	6.9	2.8	1.07	1	N
2	75	male	4.0	0.65	682	128	319	7.1	1.1	0.99	0	N
3	64	male	2.0	0.55	728	29	252	7.8	4.9	NaN	1	N
4	24	male	3.2	1.54	348	270	339	6.7	2.0	1.05	0	N

In [61]:

```
# apply() on a series- equivalent to map() for simple function
df['TB_squared'] = df['TB'].apply(lambda x : x * 2)
df[['TB', 'TB_squared']].head()
```

Out[61]:

	TB	TB_squared
0	5.8	11.6
1	5.8	11.6
2	4.0	8.0
3	2.0	4.0
4	3.2	6.4

In [62]:

```
# apply() on DataFrame, row-wise (axis=1)- function receives full row
def liver_risk(row):
    return 'High' if (row['TB'] > 3 or row['sgpt'] > 150) else 'Low'
df['Risk'] = df.apply(liver_risk, axis=1)
df[['TB', 'sgpt', 'Risk']].head()
```

Out[62]:

	TB	sgpt	Risk
0	5.8	243	High
1	5.8	204	High
2	4.0	128	High
3	2.0	29	Low
4	3.2	270	High

In [63]:

```
# apply() on DataFrame, column-wise (axis=0, the default)- function receives full row
df[['Age', 'TB', 'DB']].apply(lambda col: col.max() - col.min())
```

Out[63]:

```
Age      86.0
TB       11.0
DB        5.9
dtype: float64
```

In [64]:

```
# DataFrame.map() - element wise transform across the WHOLE frame
df[['Age', 'TB', 'DB']].map(lambda x: round(x, 0))
```

```
Out[64]:
```

	Age	TB	DB
0	55	6.0	2.0
1	18	6.0	1.0
2	75	4.0	1.0
3	64	2.0	1.0
4	24	3.0	2.0
...	...	...	...
198	47	7.0	1.0
199	35	0.0	1.0
200	86	2.0	1.0
201	86	2.0	1.0
202	36	2.0	1.0

203 rows × 3 columns

10. Missing Values

```
In [69]: df.isnull().sum()    # count of missing values per column
```

```
Out[69]: Age                0
Gender                0
TB                   0
DB                   0
Alkpos               0
sgpt                 0
sgot                 0
TP                   0
ALB                  0
AG                    4
LiverPatient        0
dtype: int64
```

```
In [70]: df.isna().sum()    # isna() is an alias for isnull() - identical behavior
```

```
Out[70]: Age                0
Gender                0
TB                   0
DB                   0
Alkpos               0
sgpt                 0
sgot                 0
TP                   0
ALB                  0
AG                    4
LiverPatient        0
dtype: int64
```

```
In [71]: df.notnull().sum()  # inverse : count of non-missing values per column
```



```
Out[71]: Age          203
        Gender       203
        TB           203
        DB           203
        Alkpos       203
        sgpt         203
        sgot         203
        TP           203
        ALB          203
        AG           199
        LiverPatient 203
        dtype: int64
```

```
In [72]: # dropna(): remove rows with ANY missing value
print("Before:", df.shape)
print("After dropna():", df.dropna().shape)
```

Before: (203, 11)

After dropna(): (199, 11)

```
In [73]: # fillna() with a constant
df['AG'].fillna(1.0).isnull().sum()
```

Out[73]: 0

```
In [74]: # fillna() with GROUPEd statistic- more accurate than a single global fill value
df['AG'] = df.groupby('Gender')['AG'].transform(lambda x: x.fillna(x.mean()))
df['AG'].isnull().sum()
```

Out[74]: 0

```
In [3]: # interpolate(): fill gaps using linear(or other) interpolation - useful for ord
import numpy as np
import pandas as pd
demo = pd.Series([1.0, np.nan, np.nan, 4.0, 5.0])
print(demo.interpolate())
```

0 1.0

1 2.0

2 3.0

3 4.0

4 5.0

dtype: float64

11. Duplicate Data

```
In [77]: df.duplicated().sum()    # total number of duplicate rows
```

Out[77]: 3

```
In [78]: df[df.duplicated()]    # view the duplicate rows themselves
```

```
Out[78]:
```

	Age	Gender	TB	DB	Alkpos	sgpt	sgot	TP	ALB	AG	LiverPatient
200	86	female	2.3	1.02	537	265	116	7.2	1.8	0.61	0
201	86	female	2.3	1.02	537	265	116	7.2	1.8	0.61	0
202	36	male	2.1	1.20	177	218	29	4.9	2.3	0.76	1

```
In [79]: df.duplicated(subset=['Age', 'Gender']).sum() # duplicate based on a SUBSET of c
```

```
Out[79]: 94
```

```
In [81]: print("Before:", df.shape)
df_clean = df.drop_duplicates()
print("After drop_duplicates():", df_clean.shape)
```

Before: (203, 11)

After drop_duplicates(): (200, 11)

```
In [82]: # keep='last' keep the LAST occurrence insted of a first
df.drop_duplicates(keep='last').shape
```

```
Out[82]: (200, 11)
```

12.String Operations (.str accessor)

```
In [83]: df['Gender'].str.lower().head()
```

```
Out[83]: 0    male
1    male
2    male
3    male
4    male
Name: Gender, dtype: object
```

```
In [84]: # strip() removes leading / trailing whitespace
messy = pd.Series([' Male', 'Female ', ' Male '])
messy.str.strip()
```

```
Out[84]: 0    Male
1    Female
2    Male
dtype: object
```

```
In [85]: df['Gender'].str.replace('Male', 'M', regex=False).head()
```

```
Out[85]: 0    male
1    male
2    male
3    male
4    male
Name: Gender, dtype: object
```

```
In [86]: # split() - break a string column into parts
names = pd.Series(['Asha Rao', 'Rohit Mehta', 'Mei Lin'])
names.str.split(' ')
```

```
Out[86]: 0      [Asha, Rao]
         1      [Rohit, Mehta]
         2      [Mei, Lin]
         dtype: object
```

```
In [87]: names.str.split(' ', expand=True) # expand=True returns a DataFrame of split p
```

```
Out[87]:
```

	0	1
0	Asha	Rao
1	Rohit	Mehta
2	Mei	Lin

```
In [89]: # contains() - boolean mask for substring matching
df[df['Gender'].str.contains('Fem')]
df.head(3)
```

```
Out[89]:
```

	Age	Gender	TB	DB	Alkpos	sgpt	sgot	TP	ALB	AG	LiverPatient
0	55	male	5.8	1.57	559	243	157	7.1	3.9	0.41	1
1	18	male	5.8	0.89	74	204	72	6.9	2.8	1.07	1
2	75	male	4.0	0.65	682	128	319	7.1	1.1	0.99	0

```
In [91]: # extract() - pull out pattern using a regex capture group
codes = pd.Series(['ID-001', 'ID-002', 'ID-103'])
codes.str.extract(r'ID-(\d+)')
```

```
Out[91]:
```

	0
0	001
1	002
2	103

13. Date and Time

```
In [92]: # to_datetime() converts strings / objects into proper datetime64 dtype
dates = pd.to_datetime(['2024-01-15', '2024-02-20', '2024-03-05'])
print(dates)
print(dates.dtype)
```

```
DatetimeIndex(['2024-01-15', '2024-02-20', '2024-03-05'], dtype='datetime64[ns]',
freq=None)
datetime64[ns]
```

```
In [97]: # Attach a synthetic 'AdmitDate' column to df for demonstration
rng = np.random.default_rng(7)
df['AdmitDate'] = (
    pd.to_datetime('2024-01-01')+
    pd.to_timedelta(rng.integers(0,200, len(df)), unit='D')
)
df[['Age', 'AdmitDate']].head()
```

Out[97]:

	Age	AdmitDate
0	55	2024-07-07
1	18	2024-05-05
2	75	2024-05-16
3	64	2024-06-28
4	24	2024-04-25

```
In [98]: # .dt accessor - extract date component
df['AdmitMonth'] = df['AdmitDate'].dt.month
df['AdmitDay'] = df['AdmitDate'].dt.day_name()
df[['AdmitDate', 'AdmitMonth', 'AdmitDay']].head()
```

Out[98]:

	AdmitDate	AdmitMonth	AdmitDay
0	2024-07-07	7	Sunday
1	2024-05-05	5	Sunday
2	2024-05-16	5	Thursday
3	2024-06-28	6	Friday
4	2024-04-25	4	Thursday

```
In [100... # date_range() - generate a sequence of dates
pd.date_range(start='2024-01-01', periods=5, freq='D')
```

```
Out[100... DatetimeIndex(['2024-01-01', '2024-01-02', '2024-01-03', '2024-01-04',
                '2024-01-05'],
                dtype='datetime64[ns]', freq='D')
```

```
In [101... pd.date_range(start='2024-01-01', periods=5, freq='ME')
```

```
Out[101... DatetimeIndex(['2024-01-31', '2024-02-29', '2024-03-31', '2024-04-30',
                '2024-05-31'],
                dtype='datetime64[ns]', freq='ME')
```

```
In [104... # resample() - group time-series data into fixed time buckets
ts = df.set_index('AdmitDate').sort_index()
monthly_admissions = ts['Age'].resample('ME').count()
monthly_admissions
```

```
Out[104... AdmitDate
2024-01-31    33
2024-02-29    22
2024-03-31    30
2024-04-30    37
2024-05-31    30
2024-06-30    29
2024-07-31    22
Freq: ME, Name: Age, dtype: int64
```

```
In [105... ts['TB'].resample('ME').mean().head()
```

```
Out[105... AdmitDate
2024-01-31    1.906061
2024-02-29    1.868182
2024-03-31    1.920000
2024-04-30    2.021622
2024-05-31    2.340000
Freq: ME, Name: TB, dtype: float64
```

14. GroupBy - Split, Apply, Combine

```
In [106... # Split + apply (mean) + combine in one line
df.groupby('Gender')['Age'].mean()
```

```
Out[106... Gender
female    47.340000
male      44.751634
Name: Age, dtype: float64
```

```
In [107... # size() - count of rows per group
df.groupby('Gender').size()
```

```
Out[107... Gender
female      50
male       153
dtype: int64
```

```
In [111... # get_group() - pull out a single group as its own DataFrame
df.groupby('Gender').get_group('female').head()
```

```
Out[111...   Age  Gender  TB  DB  Alkpos  sgpt  sgot  TP  ALB  AG  LiverPatient  AdmitDate
5   86  female  2.3  1.02    537   265   116  7.2   1.8  0.61             0  2024-06-01
7   78  female  0.3  2.33    479   235   322  7.9   5.1  0.94             0  2024-02-11
9   27  female  3.4  3.94    609   289   250  8.3   3.6  0.63             1  2024-03-01
10   6  female  2.0  0.94    500    90   268  6.5   3.6  0.82             1  2024-02-21
14  33  female  0.6  1.30    313    68    63  7.3   3.2  0.45             1  2024-04-01
```

```
In [112... # agg()- apply multiple aggeration functions at once
df.groupby('Gender')['Age'].agg(['mean', 'min', 'max', 'count'])
```

```
Out[112...   mean  min  max  count
Gender
female  47.340000    5   86     50
male    44.751634    4   90    153
```

```
In [113... # agg() with different functions per column
df.groupby('Gender').agg({
    'Age': 'mean',
    'TB' : ['mean', 'max'],
```

```
'LiverPatient' : 'sum'
})
```

Out[113...

	Age	TB	LiverPatient
	mean	mean	max
Gender			sum
female	47.340000	1.712000	6.4
male	44.751634	2.147712	11.0

In [114...

```
# transform()
df['Age_group_mean'] = df.groupby('Gender')['Age'].transform('mean')
df[['Gender', 'Age', 'Age_group_mean']].head()
```

Out[114...

	Gender	Age	Age_group_mean
0	male	55	44.751634
1	male	18	44.751634
2	male	75	44.751634
3	male	64	44.751634
4	male	24	44.751634

In [115...

```
# filter()
df.groupby('Gender').filter(lambda g: g['Age'].mean() > 40).head()
```

Out[115...

	Age	Gender	TB	DB	Alkpos	sgpt	sgot	TP	ALB	AG	LiverPatient	Admit
0	55	male	5.8	1.57	559	243	157	7.1	3.9	0.410000	1	2024-0
1	18	male	5.8	0.89	74	204	72	6.9	2.8	1.070000	1	2024-0
2	75	male	4.0	0.65	682	128	319	7.1	1.1	0.990000	0	2024-0
3	64	male	2.0	0.55	728	29	252	7.8	4.9	1.053576	1	2024-0
4	24	male	3.2	1.54	348	270	339	6.7	2.0	1.050000	0	2024-0

15. Aggregation Function

In [116...

```
print("sum:", df['Age'].sum())
print("mean:", df['Age'].mean())
print("median:", df['Age'].median())
print("mode:\n", df['Gender'].mode())
```

```
sum: 9214
mean: 45.389162561576356
median: 47.0
mode:
0    male
Name: Gender, dtype: object
```

```
In [117... print("std:", df['Age'].std())
print("var:", df['Age'].var())
print("count:", df['Age'].count())
print("nunique:", df['Age'].nunique())
```

```
std: 25.893178902013187
var: 670.4567136516608
count: 203
nunique: 77
```

```
In [118... print("min:", df['Age'].min())
print("max:", df['Age'].max())
```

```
min: 4
max: 90
```

```
In [119... df['DB'].nlargest()    # top 5 values by default
```

```
Out[119... 121    5.90
165    4.70
66     4.29
9      3.94
126    3.69
Name: DB, dtype: float64
```

```
In [120... df['DB'].nsmallest()    # smallest 3 value
```

```
Out[120... 62     0.00
155     0.00
109     0.01
125     0.01
119     0.02
Name: DB, dtype: float64
```

```
In [121... df['DB'].product()
```

```
Out[121... 0.0
```

```
In [122... # value_counts
df['Gender'].value_counts()
```

```
Out[122... Gender
male      153
female    50
Name: count, dtype: int64
```

```
In [124... df['Gender'].value_counts(normalize=True)
```

```
Out[124... Gender
male      0.753695
female    0.246305
Name: proportion, dtype: float64
```

```
In [125... # combine several aggregation into a single summary table for the whole DataFrame
df.select_dtypes(include=np.number).agg(['mean', 'std', 'max', 'min']).T
```

Out[125...

	mean	std	max	min
Age	45.389163	25.893179	90.00	4.000000
TB	2.040394	1.990881	11.00	0.000000
DB	1.000591	0.908387	5.90	0.000000
Alkpos	389.433498	202.125878	748.00	60.000000
sgpt	155.320197	85.254488	298.00	12.000000
sgot	171.369458	102.553953	347.00	12.000000
TP	6.623645	1.010358	8.90	3.600000
ALB	3.069951	0.814250	5.20	1.000000
AG	1.054953	0.402613	2.04	-0.150000
LiverPatient	0.714286	0.452871	1.00	0.000000
AdmitMonth	3.906404	1.936136	7.00	1.000000
Age_group_mean	45.389163	1.117977	47.34	44.751634

In []:

NumPy - The Complete Beginner-to-Advanced Guide

```
In [1]: import numpy as np
print(np.__version__)
print(np.version.version)
```

1.26.0

1.26.0

```
In [2]: import sys
import numpy as np

lst = list(range(0,300))
ar = np.arange(300)
print('list taking memory =', sys.getsizeof(lst) * len(lst))
print('numpy array memory =', ar.size * ar.itemsize)
```

list taking memory = 736800

numpy array memory = 1200

Creating Arrays

```
In [4]: import numpy as np

# 0-D array
a0 = np.array(324)
print(a0 ,type(a0), a0.ndim)

# 1-D array from a tuple
```



```

a1 = np.array((3,34,4,12,45))
print(a1,type(a1))

# 2-D array from nested tuple
a2 = np.array(((2,4,4), (2,5,3)))
print(a2)
print(a2.ndim)

# 2-D array from nested Lists
a22 = np.array([[2,4,5], [2,7,8]])
print(a22.ndim)

```

```

324 <class 'numpy.ndarray'> 0
[ 3 34  4 12 45] <class 'numpy.ndarray'>
[[2 4 4]
 [2 5 3]]
2
2

```

```

In [5]: # np.arange()
a = np.arange(10,100,10)
print(a)
print(a.ndim)

```

```

[10 20 30 40 50 60 70 80 90]
1

```

```

In [6]: # np.linspace()
import numpy as np
a = np.linspace(0, 100, 10)
print(a)

```

```

[ 0.          11.11111111  22.22222222  33.33333333  44.44444444
 55.55555556  66.66666667  77.77777778  88.88888889 100.         ]

```

```

In [7]: # np.zeros() / np.ones()
a = np.zeros(5)
print(a)

a1 = np.ones(6)
print(a1)

# 2-D versions
A = np.zeros((3,4))
B = np.ones((5,4))
print(A)
print(B)

```

```

[0. 0. 0. 0. 0.]
[1. 1. 1. 1. 1. 1.]
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]

```

```

In [8]: # np.concatenate()
A = np.zeros((2,3))
B = np.ones((4,3))

```

```
result = np.concatenate([A,B])
print(result)
```

```
[[0. 0. 0.]
 [0. 0. 0.]
 [1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
```

```
In [9]: # .astype()
A = np.zeros((2,3))
print(A.astype(np.uint16))
```

```
[[0 0 0]
 [0 0 0]]
```

```
In [10]: # np.random.randint()
sample = np.random.randint(1, 100, 5)
print(sample)
```

```
[12 25 97 88 26]
```

Array Attributes - ndim,shape,size,dtype,itemsize

```
In [13]: import numpy as np
a = np.array([[1,2,3,4], [1,3,4,5]])
print(a)
print('ndim :', a.ndim)
print('shape :', a.shape)
print('size :', a.size)
print('dtype :', a.dtype)
print('itemsize :', a.itemsize)
print('nbytes :', a.nbytes)
```

```
[[1 2 3 4]
 [1 3 4 5]]
ndim : 2
shape : (2, 4)
size : 8
dtype : int32
itemsize : 4
nbytes : 32
```

Indexing - Accessing a Single Value

```
In [14]: a = np.arange(10,100,9)
print(a)
print(a[2])    # third element (index 2)
print(a[-1])   # last element, negative indexing works just like lists
```

```
[10 19 28 37 46 55 64 73 82 91]
28
91
```

```
In [16]: # 2-D indexing example:
mat = np.array([[1,2,3], [4,5,6]])
print(mat[0,2])    # row 0, column 2 -> 3
print(mat[1,1])    # row 1, column 1 -> 5
```

```
# Note: mat[1][1] also works(chained indexing) but mat[1,1] is preferred-faster,m
```

3
5

Slicing - Accessing Multiple Values

In [17]: `import numpy as np`

```
a = np.arange(10,100,9)
print(a)
print(a[1:8:2])
```

```
[10 19 28 37 46 55 64 73 82 91]
[19 37 55 73]
```

In [18]: `mat = np.array([[1,2,3,4],
 [5,6,7,8],
 [9,10,11,12]])`
`print(mat[0:2, 1:3])`
`print(mat[:, 0])`
`print(mat[1, :])`

```
[[2 3]
 [6 7]]
[1 5 9]
[5 6 7 8]
```

In [19]: `a = np.arange(10)`
`b = a[2:5] # b is VIEW into a's memory not a seperate array`
`b[0] = 999 # modifying b`
`print(a)`

```
[ 0  1 999  3  4  5  6  7  8  9]
```

Fancy Indexing - Accessing multiple values by an index list

In [20]: `a = np.arange(10,100,9)`
`print(a)`
`print(a[[4,2,2]]) # pulls index 4, then 2 then 2 again -- order and repeats are`

```
[10 19 28 37 46 55 64 73 82 91]
[46 28 28]
```

Boolean (Mask) indexing -Accessing values by condition

```
In [21]: a = np.arange(10,100,9)
print(a)

mask = (a % 2 == 0)
print(mask)          # a boolean array: true where the condition holds

print(a[mask])       # filtered result--- only even values
print(a[a % 2 == 0]) # same things written in one line(the ido

[10 19 28 37 46 55 64 73 82 91]
[ True False  True False  True False  True False  True False]
[10 28 46 64 82]
[10 28 46 64 82]
```

```
In [22]: a = np.arange(10,100,9)
print(a[(a > 20) & (a < 70)])
print(a[(a < 20) | (a > 80)])

[28 37 46 55 64]
[10 19 82 91]
```

Mutability - Replace Add(Append),Remove(Delete)

```
In [23]: # Replacing a value
import numpy as np

a = np.arange(10,20,2)
print(a)
print(a[2])          # 14
a[2] = 144           # direct assignment -- mutates the array in place
print(a)

[10 12 14 16 18]
14
[ 10  12 144  16  18]
```

```
In [24]: # Adding a value
a = np.array([10,12,144,16,18])
b = np.append(a, [111, 222])
print(b)
print(a)             # a is UNCHANGED--- append returned a new array

[ 10  12 144  16  18 111 222]
[ 10  12 144  16  18]
```

```
In [25]: # Removing value
b = np.array([10,12,144,16,18,111,222])
c = np.delete(b, 2)   # delete the value AT index 2
print(c)
d = np.delete(c, [1,2])
print(d)

[ 10  12  16  18 111 222]
[ 10  18 111 222]
```

Sorting - np.sort()

```
In [26]: data = np.array([23,32,4,5,6,23,54,23])
sorted_data = np.sort(data)
print(sorted_data)
print(data)

data.sort()
print(data)
```

```
[ 4  5  6 23 23 23 54]
[23 32  4  5  6 23 54 23]
[ 4  5  6 23 23 23 54]
```

```
In [27]: # 2-D sorting and the axis parameter
mat = np.array([[3,1,2], [9,7,8]])
print(np.sort(mat, axis=1))      # sort each ROW independently
print(np.sort(mat, axis=0))      # sort each COLUMN independently
```

```
[[1 2 3]
 [7 8 9]]
[[3 1 2]
 [9 7 8]]
```

```
In [28]: data = np.array([23,32,4,5])
print(np.argsort(data))
```

```
[2 3 0 1]
```

Copy vs View - .copy()

```
In [29]: import numpy as np
a = np.array([1,23,4])
b = a.copy()
print(a)
print(b)

b[0] = 999
print('a (original):', a)      # UNCHANGED
print('b (copy):', b)          # Changed independently
```

```
[ 1 23  4]
[ 1 23  4]
a (original): [ 1 23  4]
b (copy): [999 23  4]
```

```
In [30]: # THE BUG: forgetting that slicing gives a value
```

```
original = np.array([10,20,30,40,50])
subset = original[1:3]
subset[:] = 0
print(original)
```

```
[10  0  0 40 50]
```

Rounding - np.around()

```
In [31]: a = np.array([1.2423,5.689,7.09456])
print(np.around(a, 2))
```

```
[1.24 5.69 7.09]
```

Transpose

```
In [32]: b = np.array([1,2,34,5])
print(b.transpose)
print(b.transpose())
print(b.T)
```

```
<built-in method transpose of numpy.ndarray object at 0x000001C72C7D7F90>
[ 1  2 34  5]
[ 1  2 34  5]
```

```
In [33]: mat = np.array([[1,2,3], [4,5,6]])
print(mat.shape)
print(mat.T)
print(mat.T.shape)
```

```
(2, 3)
[[1 4]
 [2 5]
 [3 6]]
(3, 2)
```

Aggregation - np.sum() and friends

```
In [34]: arr = np.array([[1,2,3], [4,5,6]])
total_sum = np.sum(arr)
print(total_sum)
```

```
21
```

```
In [35]: arr = np.array([[1,2,3], [4,5,6]])
print('sum :', np.sum(arr))
print('mean :', np.mean(arr))
print('std :', np.std(arr))
print('max :', np.max(arr))
print('argmax :', np.argmax(arr))
```

```
sum : 21
mean : 3.5
std : 1.707825127659933
max : 6
argmax : 5
```

EXTENDED TOPICS (Essential for Interviews,Beyond the PDF)

```
In [36]: # Reshaping - .reshape()
import numpy as np

a = np.arange(12)
print(a)

b = a.reshape(3,4)
print(b)
```

```
c = a.reshape(2,2,3)  # reshape into 3-D
print(c)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[[ 0  1  2]
  [ 3  4  5]]

 [[ 6  7  8]
  [ 9 10 11]]]
```

```
In [37]: a = np.arange(12)
print(a.reshape(3,-1))
print(a.reshape(-1,4))
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

Faltten vs Ravel

```
In [38]: mat = np.array([[1,2,3], [4,5,6]])
flat = mat.flatten()
rav = mat.ravel()

print(flat)
print(rav)

rav[0] = 999      # ravel is a view -- this will affect mat
print(mat)

flat[0] = -1      # flatten is a copy -- this will not affect mat
print(mat)
```

```
[1 2 3 4 5 6]
[1 2 3 4 5 6]
[[999  2  3]
 [  4  5  6]]
[[999  2  3]
 [  4  5  6]]
```

Broadcasting

```
In [39]: # Simplest case: scalar broadcasting
a = np.array([1,2,3])
print(a + 10)

# Vector + Matrix broadcasting
mat = np.array([[1,2,3],
                 [4,5,6],
                 [7,8,9]])
row = np.array([10,20,30])
print(mat + row)
```

```
[11 12 13]
[[11 22 33]
 [14 25 36]
 [17 28 39]]
```

```
In [40]: a = np.array([[1,2,3], [4,5,6]])
         b = np.array([1,2])
         try:
             print(a + b)
         except ValueError as e:
             print("error:", e)
```

error: operands could not be broadcast together with shapes (2,3) (2,)

Vectorization

```
In [41]: import time

         L = list(range(1_000_000))
         a = np.arange(1_000_000)

         start = time.time()
         squared_list = [i**2 for i in L]
         print("python loop:", time.time() - start, "seconds")

         start = time.time()
         squared_arr = a ** 2
         print("numpy vectorized:", time.time() - start, "seconds")
```

python loop: 0.2568514347076416 seconds

numpy vectorized: 0.005001544952392578 seconds

Universal Function(ufuncs)

```
In [42]: a = np.array([1,4,9,16])
         print(np.sqrt(a))
         print(np.exp(a))
         print(np.log(a))
         print(np.sin(a))

         # Arithmetic ufuncs
         b = np.array([1,2,3,4])
         print(np.add(a,b))
         print(np.multiply(a,b))
```

```
[1.  2.  3.  4.]
[2.71828183e+00  5.45981500e+01  8.10308393e+03  8.88611052e+06]
[0.          1.38629436  2.19722458  2.77258872]
[ 0.84147098 -0.7568025   0.41211849 -0.28790332]
[ 2   6 12 20]
[ 1   8 27 64]
```

The Axis Parameter(Explained Visually)

```
In [43]: mat = np.array([[1,2,3],
                        [4,5,6]])
         print('no axis (whole array):', np.sum(mat))
```



```
print('axis=0 (sum DOWN each column: ', np.sum(mat, axis=0))
print('axis=1 (sum ACROSS each row): ', np.sum(mat, axis=1))
```

```
no axis (whole array): 21
axis=0 (sum DOWN each column: [5 7 9]
axis=1 (sum ACROSS each row): [ 6 15]
```

Stacking Arrays

```
In [44]: import numpy as np

a = np.array([1,2,3])
b = np.array([4,5,6])
print(np.vstack([a,b]))
print(np.hstack([a,b]))
print(np.stack([a,b]))
print(np.stack([a,b], axis=1))
```

```
[[1 2 3]
 [4 5 6]]
[1 2 3 4 5 6]
[[1 2 3]
 [4 5 6]]
[[1 4]
 [2 5]
 [3 6]]
```

Splitting Arrays

```
In [45]: a = np.arange(9)
print(np.split(a,3))

mat = np.arange(16).reshape(4,4)
print(np.vsplit(mat, 2))
print(np.hsplit(mat, 2))
```

```
[array([0, 1, 2]), array([3, 4, 5]), array([6, 7, 8])]
[array([[0, 1, 2, 3],
       [4, 5, 6, 7]]), array([[ 8,  9, 10, 11],
       [12, 13, 14, 15]])]
[array([[ 0,  1],
       [ 4,  5],
       [ 8,  9],
       [12, 13]]), array([[ 2,  3],
       [ 6,  7],
       [10, 11],
       [14, 15]])]
```

Random Module - Deeper Dive

```
In [46]: np.random.seed(42)
print(np.random.rand(3))
print(np.random.randn(3))
print(np.random.choice([10,20,30,40], size=3))
print(np.random.permutation(5))
```

```
[0.37454012 0.95071431 0.73199394]
[-1.11188012 0.31890218 0.27904129]
[40 30 20]
[0 2 3 1 4]
```

Statistics with NumPy

```
In [47]: data = np.array([12,15,14,10,18,20,22,25,30,100])
print('mean :', np.mean(data))
print('median :', np.median(data))
print('std dev :', np.std(data))
print('variance :', np.var(data))
print('25th pct :', np.percentile(data, 25))
print('75th pct :', np.percentile(data, 75))
print('correlation matrix :')
x = np.array([1,2,3,4,5])
y = np.array([2,4,5,4,5])
print(np.corrcoef(x,y))
```

```
mean : 26.6
median : 19.0
std dev : 25.144383070578606
variance : 632.24
25th pct : 14.25
75th pct : 24.25
correlation matrix :
[[1.          0.77459667]
 [0.77459667 1.          ]]
```

Matrix Operation

```
In [48]: A = np.array([[1,2], [3,4]])
B = np.array([[5,6], [7,8]])
print('elementwise (A * B):')
print(A * B)
print('matrix manipulation( A @ B):')
print(A @ B)
```

```
elementwise (A * B):
[[ 5 12]
 [21 32]]
matrix manipulation( A @ B):
[[19 22]
 [43 50]]
```

NumPy Practice - 50 Coding Questions(Easy --> Hard)

1. Create a 1-D array of numbers 1 to 20.

```
In [1]: import numpy as np
# Create 1-D array from 1 to 20
```

```
arr = np.arange(1, 21)
print(arr)
```

```
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]
```

2. Create a 5*5 array of all zeros.

```
In [6]: import numpy as np
#Create 5x5 array of zeros
arr = np.zeros((5, 5))
print(arr)
```

```
[[0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]]
```

3. Create a 4*4 identity matrix.

```
In [7]: import numpy as np
# Create 4x4 identity matrix
arr = np.eye(4)
print(arr)
```

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

4. Create an array of 10 evenly spaced numbers between 0 and 1.

```
In [8]: import numpy as np
# Create 10 evenly spaced numbers between 0 and 1
arr = np.linspace(0, 1, 10)
print(arr)
```

```
[0.          0.11111111 0.22222222 0.33333333 0.44444444 0.55555556
 0.66666667 0.77777778 0.88888889 1.          ]
```

5. Reverse a given 1-D array.

```
In [9]: import numpy as np
# Original array
arr = np.array([1, 2, 3, 4, 5])
# Reverse the array
rev_arr = arr[::-1]
print("Original:", arr)
print("Reversed:", rev_arr)
```

```
Original: [1 2 3 4 5]
Reversed: [5 4 3 2 1]
```

6. Find the shape,size,and dtype of a given array.

```
In [10]: import numpy as np
# Create sample array
arr = np.array([[1, 2, 3], [4, 5, 6]])
# Find properties
print("Array:\n", arr)
print("Shape:", arr.shape)
print("Size:", arr.size)
print("Data Type:", arr.dtype)
```

```
Array:
[[1 2 3]
 [4 5 6]]
Shape: (2, 3)
Size: 6
Data Type: int32
```

7. Replace all odd numbers in an array with -1.

```
In [11]: arr = np.arange(1, 11)
arr[arr % 2 != 0] = -1
print(arr)
```

```
[-1  2 -1  4 -1  6 -1  8 -1 10]
```

8. Find the sum, mean, and max of an array.

```
In [12]: arr = np.array([1,2,3,4,5])
print("Sum:", np.sum(arr))
print("Mean:", np.mean(arr))
print("Max:", np.max(arr))
```

```
Sum: 15
Mean: 3.0
Max: 5
```

9. Create a 3x3 array of random integers between 1 and 100.

```
In [13]: arr = np.random.randint(1, 101, (3,3))
print(arr)
```

```
[[51  5 92]
 [83 39 21]
 [ 2 29 37]]
```

10. Stack two 1-D arrays vertically and horizontally

```
In [14]: a = np.array([1,2,3])  
b = np.array([4,5,6])  
print(np.vstack((a,b)))  
print(np.hstack((a,b)))
```

```
[[1 2 3]  
 [4 5 6]]  
[1 2 3 4 5 6]
```

11. Extract every 3rd element from an array.

```
In [15]: arr = np.arange(1, 16)  
print(arr[::3])
```

```
[ 1  4  7 10 13]
```

12. Find common elements between two arrays.

```
In [16]: a = np.array([1,2,3,4])  
b = np.array([3,4,5,6])  
print(np.intersect1d(a, b))
```

```
[3 4]
```

13. Convert a list of temperatures in Celsius to Fahrenheit using vectorized math.

```
In [17]: c = np.array([0, 20, 30, 40])  
f = (c * 9/5) + 32  
print(f)
```

```
[ 32.  68.  86. 104.]
```

14. Count the number of even numbers in an array.

```
In [18]: arr = np.array([1,2,3,4,5,6])  
print(np.sum(arr % 2 == 0))
```

```
3
```

15. Create an array and compute its transpose.

```
In [19]: arr = np.array([[1,2,3],[4,5,6]])
print(arr.T)
```

```
[[1 4]
 [2 5]
 [3 6]]
```

16. Normalize an array to the range [0, 1] using broadcasting.

```
In [20]: import numpy as np
arr = np.array([10, 20, 30, 40, 50])
norm = (arr - arr.min()) / (arr.max() - arr.min())
print(norm)
```

```
[0.  0.25 0.5  0.75 1.  ]
```

17. Find the indices of all elements greater than the array's mean.

```
In [21]: arr = np.array([1,2,3,4,5])
mean = np.mean(arr)
print(np.where(arr > mean))
```

```
(array([3, 4], dtype=int64),)
```

18. Create a checkerboard pattern (8x8) of 0s and 1 using slicing.

```
In [22]: arr = np.zeros((8,8), dtype=int)
arr[1::2, ::2] = 1
arr[:, 1::2] = 1
print(arr)
```

```
[[0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]]
```

19. Compute the row-wise and column-wise sum of a 2-D matrix.

```
In [23]: arr = np.array([[1,2,3],[4,5,6]])
print("Row sum:", np.sum(arr, axis=1))
print("Column sum:", np.sum(arr, axis=0))
```

Row sum: [6 15]

Column sum: [5 7 9]

20. Replace all NaN values in an array with the column mean.

```
In [24]: arr = np.array([[1, np.nan], [3, 4]])
col_mean = np.nanmean(arr, axis=0)
inds = np.where(np.isnan(arr))
arr[inds] = np.take(col_mean, inds[1])
print(arr)
```

[[1. 4.]

[3. 4.]]

21. Find the second-largest value in an array without sorting the whole array.

```
In [25]: arr = np.array([10, 20, 4, 45, 99])
print(np.partition(arr, -2)[-2])
```

45

22. Given two matrices, check if they can be multiplied, then multiply them.

```
In [30]: A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])
if A.shape[1] == B.shape[0]:
    print(np.dot(A,B))
else:
    print("Cannot multiply")
```

[[19 22]

[43 50]]

23. Implement min-max scaling on a feature matrix (axis=0).

```
In [31]: arr = np.array([[1,2],[3,4],[5,6]])
scaled = (arr - arr.min(axis=0)) / (arr.max(axis=0) - arr.min(axis=0))
print(scaled)
```

```
[[0.  0. ]
 [0.5 0.5]
 [1.  1. ]]
```

24. Generate a random matrix and compute its inverse and determinant.

```
In [32]: arr = np.random.randint(1,10,(3,3))
print("Matrix:\n", arr)
print("Determinant:", np.linalg.det(arr))
print("Inverse:\n", np.linalg.inv(arr))
```

```
Matrix:
[[8 1 5]
 [3 3 1]
 [1 9 5]]
Determinant: 154.00000000000006
Inverse:
[[ 0.03896104  0.25974026 -0.09090909]
 [-0.09090909  0.22727273  0.04545455]
 [ 0.15584416 -0.46103896  0.13636364]]
```

25. Given an array of exam scores, bucket them into grade categories using boolean masks (A/B/C/F).

```
In [33]: scores = np.array([85, 72, 60, 40])
grades = np.empty(scores.shape, dtype=str)
grades[scores >= 80] = 'A'
grades[(scores >= 60) & (scores < 80)] = 'B'
grades[(scores >= 50) & (scores < 60)] = 'C'
grades[scores < 50] = 'F'
print(grades)
```

```
['A' 'B' 'B' 'F']
```

26. Flatten a 3-D array and confirm whether the result is a view or copy

```
In [34]: arr = np.arange(8).reshape(2,2,2)
flat = arr.ravel() # view
print(flat)
# Check
flat[0] = 100
print(arr) # changes reflect → it's a view
```

```
[0 1 2 3 4 5 6 7]
[[[100  1]
 [ 2  3]]

 [[ 4  5]
 [ 6  7]]]
```


27. Compute the moving average of a 1-D array with a window size of 3.

```
In [35]: arr = np.array([1,2,3,4,5])
ma = np.convolve(arr, np.ones(3)/3, mode='valid')
print(ma)
```

```
[2. 3. 4.]
```

28. Implement one-hot encoding for an array of class labels using NumPy only.

```
In [36]: import numpy as np
labels = np.array([0, 1, 2, 1])
one_hot = np.eye(np.max(labels)+1)[labels]
print(one_hot)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]
 [0. 1. 0.]]
```

29. Given a confusion matrix, compute accuracy using only NumPy operations.

```
In [37]: cm = np.array([[5, 2],
[1, 7]])
accuracy = np.trace(cm) / np.sum(cm)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.8
```

30. Shuffle a feature matrix x and label vector y together using the same permutation.

```
In [38]: x = np.array([[1,2],[3,4],[5,6]])
y = np.array([0,1,0])
perm = np.random.permutation(len(x))
x_shuffled = x[perm]
y_shuffled = y[perm]
print(x_shuffled)
print(y_shuffled)
```

```
[[1 2]
 [3 4]
 [5 6]]
[0 1 0]
```

31. Compute pairwise Euclidean distances between two sets of points (no loops).

```
In [39]: A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])
dist = np.sqrt(((A[:, np.newaxis] - B)**2).sum(axis=2))
print(dist)
```

```
[5.65685425 8.48528137]
[2.82842712 5.65685425]]
```

32. Implement the sigmoid and ReLU functions and apply them to a random array.

```
In [40]: arr = np.random.randn(5)
sigmoid = 1 / (1 + np.exp(-arr))
relu = np.maximum(0, arr)
print("Sigmoid:", sigmoid)
print("ReLU:", relu)
```

```
Sigmoid: [0.73292161 0.51131132 0.40193695 0.33287794 0.39189111]
ReLU: [1.00949652 0.04525301 0.          0.          0.          ]
```

33. Split a dataset array into 80/20 train/test using slicing after shuffling.

```
In [41]: data = np.arange(10)
np.random.shuffle(data)
split = int(0.8 * len(data))
train = data[:split]
test = data[split:]
print("Train:", train)
print("Test:", test)
```

```
Train: [4 3 0 8 1 7 6 9]
Test: [2 5]
```

34. Given an array of RGB pixel values, convert it to grayscale using a weighted sum.

```
In [42]: rgb = np.array([[255, 0, 0],
[0, 255, 0],
[0, 0, 255]])
gray = np.dot(rgb, [0.299, 0.587, 0.114])
print(gray)
```

```
[ 76.245 149.685 29.07 ]
```

35. Find duplicate rows in a 2-D array.

```
In [43]: arr = np.array([[1,2],
[3,4],
[1,2],
[5,6]])
_, idx = np.unique(arr, axis=0, return_index=True)
duplicates = np.delete(arr, idx, axis=0)
print(duplicates)
```

```
[[1 2]]
```

36. Implement a simple linear regression using the normal equation (no Scikit-learn).

```
In [44]: import numpy as np
X = np.array([[1],[2],[3],[4]])
y = np.array([2,4,6,8])
X_b = np.c_[np.ones((len(X),1)), X] # add bias
theta = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
print("Intercept & Slope:", theta)
```

```
Intercept & Slope: [0. 2.]
```

37. Implement K-Means cluster assignment step (assign each point to its nearest centroid) using only broadcasting.

```
In [45]: X = np.array([[1,2],[3,4],[5,6]])
centroids = np.array([[1,1],[5,5]])
print(labels)
dist = np.sqrt(((X[:, np.newaxis] - centroids)**2).sum(axis=2))
labels = np.argmin(dist, axis=1)
print(labels)
```

```
[0 1 2 1]
[0 1 1]
```

38. Implement PCA from scratch using np.linalg.eig On the covariance matrix.

```
In [46]: X = np.array([[1,2],[3,4],[5,6]])
X_mean = X - np.mean(X, axis=0)
cov = np.cov(X_mean, rowvar=False)
eig_val, eig_vec = np.linalg.eig(cov)
```

```
print("Eigenvalues:", eig_val)
print("Eigenvectors:\n", eig_vec)
```

```
Eigenvalues: [8. 0.]
Eigenvectors:
[[ 0.70710678 -0.70710678]
 [ 0.70710678  0.70710678]]
```

39. Implement a single forward pass of a 2-layer neural network using only NumPy (matrix multiply + sigmoid).

```
In [48]: X = np.array([[1,2]])
W1 = np.random.rand(2,3)
W2 = np.random.rand(3,1)
def sigmoid(x):
    return 1/(1+np.exp(-x))
h = sigmoid(X @ W1)
output = sigmoid(h @ W2)
print(output)
```

```
[[0.73851922]]
```

40. Implement softmax (numerically stable version, subtracting the max) for a batch of logits.

```
In [49]: x = np.array([2.0, 1.0, 0.1])
e_x = np.exp(x - np.max(x))
softmax = e_x / e_x.sum()
print(softmax)
```

```
[0.65900114 0.24243297 0.09856589]
```

41. Implement gradient descent to fit a line $y = mx + b$ to noisy data.

```
In [51]: x = np.array([1,2,3,4])
y = np.array([2,4,6,8])
m, b = 0, 0
lr = 0.01
for _ in range(1000):
    y_pred = m*x + b
    dm = (-2/len(x)) * np.sum(x*(y - y_pred))
    db = (-2/len(x)) * np.sum(y - y_pred)

    m -= lr*dm
    b -= lr*db
print(m, b)
```

```
1.9896587550255742 0.030404521305361965
```

42. Compute a sliding-window (convolution-style) sum over a 1-D signal without using np.convolve .

```
In [53]: arr = np.array([1,2,3,4,5])
k = 3

result = np.array([arr[i:i+k].sum() for i in range(len(arr)-k+1)])
print(result)
```

[6 9 12]

43. Implement batch normalization (subtract mean, divide by std, per feature) for a batch of data.

```
In [54]: X = np.array([[1,2],[3,4],[5,6]])
mean = np.mean(X, axis=0)
std = np.std(X, axis=0)
norm = (X - mean) / std
print(norm)
```

[[-1.22474487 -1.22474487]
[0. 0.]
[1.22474487 1.22474487]]

44. Given a dataset with outliers, implement an IQR based outlier removal function using only boolean masking.

```
In [55]: arr = np.array([10,12,14,100,18,20])
Q1 = np.percentile(arr, 25)
Q3 = np.percentile(arr, 75)
IQR = Q3 - Q1
filtered = arr[(arr >= Q1 - 1.5*IQR) & (arr <= Q3 + 1.5*IQR)]
print(filtered)
```

[10 12 14 18 20]

45. Implement nearest-neighbor classification (1-NN) using only NumPy distance computations.

```
In [56]: train_X = np.array([[1,2],[3,4],[5,6]])
train_y = np.array([0,1,1])
test = np.array([2,3])
dist = np.sqrt(((train_X - test)**2).sum(axis=1))
```

```
label = train_y[np.argmin(dist)]
print(label)
```

0

46. Build a simple image-patch extractor: given an image array and patch size, extract all non-overlapping patches.

```
In [57]: img = np.arange(16).reshape(4,4)
patch_size = 2
patches = [img[i:i+patch_size, j:j+patch_size]
            for i in range(0,4,patch_size)
            for j in range(0,4,patch_size)]
print(patches)
```

```
[array([[0, 1],
        [4, 5]]), array([[2, 3],
        [6, 7]]), array([[ 8,  9],
        [12, 13]]), array([[10, 11],
        [14, 15]])]
```

47. Implement cosine similarity between two sets of vectors (matrix form, no loops).

```
In [58]: A = np.array([[1,0],[0,1]])
B = np.array([[1,1],[1,0]])
dot = A @ B.T
normA = np.linalg.norm(A, axis=1)[:,None]
normB = np.linalg.norm(B, axis=1)
cos_sim = dot / (normA * normB)
print(cos_sim)
```

```
[[0.70710678 1.
  [0.70710678 0.
   ]]
```

48. Implement a basic Markov chain transition matrix and compute the stationary distribution via eigenvectors.

```
In [59]: P = np.array([[0.9,0.1],[0.5,0.5]])
eig_val, eig_vec = np.linalg.eig(P.T)
stationary = eig_vec[:, np.isclose(eig_val,1)]
stationary = stationary / stationary.sum()
print(stationary.real)
```

```
[[0.83333333]
 [0.16666667]]
```

49. Implement Mean Squared Error and Mean Absolute Error loss functions, both vectorized.

```
In [61]: import numpy as np
# Actual and Predicted values
y_true = np.array([3, -0.5, 2, 7])
y_pred = np.array([2.5, 0.0, 2, 8])
# MSE
mse = np.mean((y_true - y_pred) ** 2)
# MAE
mae = np.mean(np.abs(y_true - y_pred))
print("MSE:", mse)
print("MAE:", mae)
```

MSE: 0.375

MAE: 0.5

50. Given your own FusionNet/AMFNet PSNR computation, rewrite it as a single vectorized NumPy function that takes two image batches (batch, H, W, C) and returns one PSNR value per image in the batch —no Python-level loop over the batch dimension.

```
In [66]: import numpy as np

def batch_psnr(img1, img2, max_pixel=255.0):
    """
    Compute PSNR for a batch of images.

    Parameters:
        img1 : numpy.ndarray
            Shape = (batch, H, W, C)
        img2 : numpy.ndarray
            Shape = (batch, H, W, C)
        max_pixel : float
            Maximum possible pixel value (255 for uint8 images)

    Returns:
        numpy.ndarray
            PSNR value for each image in the batch.
            Shape = (batch,)
    """

    # Convert to float for accurate computation
    img1 = img1.astype(np.float64)
    img2 = img2.astype(np.float64)
```

```
# Mean Squared Error for each image
mse = np.mean((img1 - img2) ** 2, axis=(1, 2, 3))

# Avoid division by zero
mse = np.maximum(mse, 1e-10)

# PSNR computation
psnr = 20 * np.log10(max_pixel) - 10 * np.log10(mse)

return psnr

# Create two batches of 5 RGB images of size 128x128
batch1 = np.random.randint(0, 256, (5, 128, 128, 3), dtype=np.uint8)
batch2 = np.random.randint(0, 256, (5, 128, 128, 3), dtype=np.uint8)

psnr_values = batch_psnr(batch1, batch2)

print("PSNR for each image:")
print(psnr_values)
```

PSNR for each image:

[7.75596694 7.74789107 7.7726949 7.77219521 7.75180467]

In []: